

暨南大学

本科生课程论文

论文题目：基于卷积神经网络（CNN）和 MNIST 数据集的手写
数字识别项目

学 院：信息科学技术学院
学 系：电子工程系
专 业：电子信息工程
课程名称：人工智能
学生姓名：陈国建
学 号：2025100972
指导教师：杨梓铎

2026 年 06 月 24 日

一、课程论文来源说明

本文选题为“基于卷积神经网络（CNN）和 MNIST 数据集的手写数字识别项目”。由于涉及的参考文献较多，此处仅列举主要参考来源平台，详细参考文献在文后列出。本文理论部分主要参考了杨梓铎老师的课程讲义、Ian Goodfellow 的著作 *Deep Learning* (The MIT Press) 中有关 CNN 的章节和 ChatGPT 极其细致的解释。实验代码由本人使用 VS Code 基于 PyTorch 独立编写完成，模型架构方面，综合参考了 Bilibili 平台、CSDN 平台、PyTorch Tutorial（官方新手文档）、ChatGPT 和 Codex（严重降智版）的从基础到进阶的多种数字识别模型。在论文写作过程中，本人在历经五天憋出一千字之后使用 ChatGPT 辅助进行文章扩充和润色，但最终实验过程、数据分析和论文撰写还是在本人监控下完成。

二、摘要

卷积神经网络（Convolutional Neural Network, CNN）是人工智能领域最具代表性的模型之一，在图像分类、目标检测等计算机视觉任务中取得了显著成果。本文基于 PyTorch 深度学习框架，采用这一模型在 MNIST 数据集上复现了手写数字识别项目，并设计了对照实验，分别比较不同优化器（SGD 与 稍高级的 Adam）和不同网络架构（BasicCNN 与稍高级的 Mini-ResNet）对分类性能的影响。同时，基于个人理解，本文尝试阐述了 CNN 的核心原理，包括卷积层、池化层、激活函数及全连接层的设计思想与数学基础；最终实验结果表明，BasicCNN 模型在测试集上取得了 99.47% 的分类准确率，同时 Mini-ResNet 模型达到了 99.52%，验证了 CNN 在手写数字识别任务中的优异性能。最后，本文源码和实验结果等已分组块上传至 GitHub 仓库：<https://github.com/NoShape33/MNIST-CNN>。

三、引言

人工智能（Artificial Intelligence, AI）是当前科技领域最受关注的研究方向之一，其核心目标在于使计算机系统具备模拟人类智能行为的能力。深度学习作为

人工智能的重要分支，通过构建多层神经网络来自动学习数据的层次化特征表示，在图像识别、语音处理、自然语言理解等众多领域取得了突破性进展。

卷积神经网络（Convolutional Neural Network, CNN）是深度学习在计算机视觉领域最具代表性的模型架构。自 LeCun 等人于 1998 年提出 LeNet-5 用于手写数字识别以来，CNN 经历了从 AlexNet、VGG、GoogLeNet 到 ResNet 等一系列重要发展，其核心思想——局部连接（Local Connectivity）与权值共享（Weight Sharing）——极大地减少了模型参数数量，同时保留了图像的空间结构信息，使网络能够高效地提取从边缘、纹理到语义的多层次视觉特征。

手写数字识别是计算机视觉领域的经典入门任务，也是模式识别研究的重要基准问题。MNIST 数据集作为人工智能项目领域的 Helloworld，包含 70,000 张 28×28 像素的灰度手写数字图像，涵盖了 0 至 9 共十个类别。尽管这个项目对于大三学生来说可能过于基础，但对于本人这个大一学生来说可谓天书，而使 MNIST 数据集对新手来说较为友好，也为本人理解 CNN 的工作原理、验证模型设计思路 and 对比不同优化策略提供理想的实验平台。本文选择以 MNIST 手写数字识别为切入点，系统复现 CNN 模型的完整实验流程，深入理解卷积神经网络的原理与实现细节，为后续研究更复杂的视觉任务奠定基础。

四、算法简介

4.1 卷积神经网络基本原理

卷积神经网络是一种专门为处理网格状拓扑结构数据（如图像）而设计的前馈神经网络。与传统简单的 MLP 多层感知机不同，CNN 通过引入卷积层、池化层等特殊结构，能够更有效地解决 MLP 在处理图像时面临的两大核心问题：参数数量爆炸和空间结构信息丢失。以 MNIST 图像为例，若使用全连接网络直接将 $28 \times 28 = 784$ 个像素连接到 1,000 个隐藏神经元，仅第一层就产生约 78 万个参数；而 CNN 通过局部连接和权值共享，可以大幅缩减参数规模。

4.2 卷积层

卷积层是 CNN 的核心组件，其通过卷积核（**Kernel**）在输入特征图上滑动计算局部区域的加权和，从而实现特征提取。其具有两个关键特性：局部连接和权值共享。局部连接意味着每个输出神经元仅与输入的一个局部区域（感受野）相连，而非全连接；权值共享则指同一个卷积核在整个输入空间上使用相同的参数，使得网络能够检测出现在图像任意位置的同一模式（平移等变性）。对于输入特征图 X 和卷积核 W ，二维卷积运算的数学表达式为：

$$Y(i, j) = (X * W)(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot W(m, n)$$

其中 Y 为输出特征图， $*$ 表示卷积运算。通过堆叠多个卷积层，网络可以逐层提取从低级到高级的层次化特征：浅层卷积核倾向于检测边缘、线条等简单模式，深层卷积核则能够捕获纹理、形状乃至语义级别的抽象特征。

4.3 池化层

池化层（**Pooling Layer**）用于对特征图进行下采样，降低空间分辨率，从而减少后续层的计算量和内存消耗，同时增强网络对局部平移的不变性（**Translation Invariance**）。常用的池化方式包括最大池化（**Max Pooling**，取窗口内最大值）和平均池化（**Average Pooling**，取窗口内均值）。本文实验采用 2×2 最大池化、步长为 2，每次操作将特征图的空间尺寸减半，在保留最显著特征响应的同时实现高效降维。

4.4 激活函数

激活函数为网络引入非线性变换能力，使得多层网络可以逼近任意复杂的函数映射。根据多个平台的建议，本文采用更为基础的 **ReLU**（**Rectified Linear Unit**）激活函数，其定义为：

$$f(x) = \max(0, x)$$

相比于 Sigmoid 和 Tanh 等传统激活函数，ReLU 具有计算简单（仅需比较和取最大值）、缓解梯度消失问题等优势，是目前 CNN 中最广泛使用的激活函数之一。当输入 $x > 0$ 时梯度恒为 1，有效避免了深层网络中梯度逐层衰减的问题。

4.5 全连接层与 Softmax 分类器

在多个卷积层和池化层提取特征之后，CNN 通常将特征图展平（Flatten）为一维向量，然后输入全连接层（Fully Connected Layer）进行高阶特征组合，最后通过 Softmax 函数将输出 logits 映射为各个类别的概率分布。Softmax 函数的数学定义为：

$$P(y = j | x) = \frac{\exp(z_j)}{\sum_k \exp(z_k)}$$

其中 Z_j 为第 j 个类别的 logits 输出，分母对所有类别求和以保证输出为合法的概率分布。本文采用 Log-Softmax 与负对数似然损失（Negative Log Likelihood Loss, NLLLoss）进行训练，该组合在数学上等价于交叉熵损失（Cross Entropy Loss），且具有更好的数值稳定性。

4.6 反向传播与参数优化

CNN 的训练通过反向传播算法（Backpropagation）实现。前向传播计算网络输出与损失函数值；反向传播则利用链式法则（Chain Rule），从输出层向输入层逐层计算损失函数对各参数的梯度。参数更新采用小批量梯度下降（Mini-batch Gradient Descent）策略，每次使用一小批样本（Batch Size=64）计算梯度并更新参数，在计算效率和梯度估计稳定性之间取得平衡。本文对比了 Adam 自适应优化器与带动量（Momentum=0.9）的 SGD 优化器的训练效果：Adam 通过自适应学习率和动量机制加速收敛，而 SGD 配合 Momentum 则可能在充分训练后获得更好的泛化性能。

五、实验设计与应用场景

5.1 数据集介绍

MNIST（Modified National Institute of Standards and Technology）数据集是手写数字识别领域最经典的公开基准数据集。该数据集初始包含 70,000 张 28×28 像素的灰度手写数字图像，其中训练集 60,000 张，测试集 10,000 张。每张图像对应 0 至 9 之间的一个数字标签，各类别样本数量基本均衡（约 6,000 张/类）。MNIST 由美国国家标准与技术研究所（NIST）的原始手写数字样本经归一化处理、尺寸标准化和抗锯齿后构建，数据来源涵盖人口普查局雇员和高中学生的笔迹样本，具有较好的书写风格多样性。

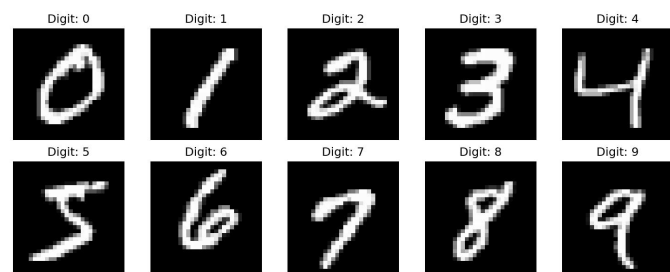


图 1 MNIST 数据集各类别样本示例（图片来源：使用 Matplotlib 从 MNIST 数据集中提取后显示）

5.2 数据预处理

数据预处理是深度学习实验的重要环节，直接影响模型的训练效率和泛化能力。本文对 MNIST 数据集的预处理包括以下步骤：

（1）归一化：将原始像素值从 $[0, 255]$ 区间线性缩放到 $[0, 1]$ （通过 ToTensor 变换），再使用 MNIST 的全局均值 $\mu=0.1307$ 和标准差 $\sigma=0.3081$ 进行 Z-Score 标准化，使数据分布接近标准正态分布 $N(0,1)$ ，有利于加速梯度下降收敛并提升数值稳定性；

（2）数据增强：对训练集样本施加 $\pm 10^\circ$ 的随机旋转（RandomRotation），模拟真实场景中书写角度的变化，有效扩充训练数据多样性，降低过拟合风险（此步骤本人交由 Codex 进行）；

（3）数据集划分：本文将初始的 60,000 张训练样本按 8:2 比例随机划分为训练集（48,000 张）和验证集（12,000 张），以固定随机种子（seed=42）确保划分可复现。验证集用于监控训练过程中的泛化能力并执行早停策略，辅助调整参数测试集（10,000 张）保持原始划分不变，仅用于最终性能评估。

5.3 实验环境与参数设置

本实验在一台华硕天选 6 Pro 笔记本上进行，GPU 为 NVIDIA GeForce RTX 5060 Laptop GPU（8 GB 显存，CUDA 版本为 13.2（最新））；软件使用 VS Code 作为 IDE，代码编写基于 Python 3.10 和 PyTorch 2.12 深度学习框架。模型训练全程采用 GPU 加速，单轮训练耗时约 8 秒，总训练耗时约 3 分钟。

实验采用的超参数配置同样参考各平台常见模型并初步调优后确定，详见表 1。

表 1 实验超参数配置

参数	取值	说明
Epochs	20	最大训练轮数，配合早停策略使用
Batch Size	64	小批量梯度下降的批次大小
Learning Rate (Adam)	0.001	Adam 优化器初始学习率
Learning Rate (SGD)	0.01	SGD 优化器初始学习率
Weight Decay	1×10^{-4}	L2 正则化系数
Patience	5	早停容忍轮数（验证损失无改善）
LR Schedule	ReduceLROnPlateau	验证损失停滞 3 轮后学习率 $\times 0.5$
Data Augmentation	RandomRotation($\pm 10^\circ$)	仅应用于训练集

5.4 模型架构

本文设计并实现了两种 CNN 模型架构进行对比实验。BasicCNN 采用三层卷积加两层全连接的经典结构设计，参考了 LeNet-5 的原始架构并引入了 Batch Normalization（BN）以加速训练收敛、稳定梯度传播；Mini-ResNet 则在基础卷积结构上引入了残差连接（Residual Connection），参考了 ResNet 的核心思想——通

过恒等映射的跳跃连接缓解深层网络中的梯度消失问题，使网络在增加深度的同时保持训练的稳定性和有效性。两种模型的详细结构配置见表 2。

表 2 两种模型架构的层级配置对比

层级	BasicCNN	输出尺寸	Mini-ResNet	输出尺寸
Input	—	1×28×28	—	1×28×28
Conv1	Conv(1→32,k3)+BN+ReLU MaxPool(2)	32×14×14	Conv(1→32,k3)+BN+ReLU	32×28×28
Block1	Conv(32→64,k3)+BN+ReLU MaxPool(2)	64×7×7	ResBlock(32→32)×2	32×28×28
Block2	Conv(64→128,k3)+BN+ReLU MaxPool(2)	128×3×3	ResBlock(32→64,s=2)×2	64×14×14
Pool	—	—	GlobalAvgPool	64×1×1
FC1	FC(1152→256)+ReLU Dropout(0.5)	256	FC(64→10)	10
Output	FC(256→10)+LogSoftmax	10	LogSoftmax	10

BasicCNN 的总参数量为 390,858 个，Mini-ResNet 为 178,634 个。值得注意的是，Mini-ResNet 虽然结构更深（含 4 个残差块共 8 个卷积层），但由于采用全局平均池化（Global Average Pooling）替代了传统的全连接展平操作，其全连接层的输入特征维度仅为 64，因此总参数量反而不足 BasicCNN 的一半（45.7%）。这一设计体现了全局平均池化在控制模型复杂度方面的显著优势，也是现代 CNN 架构（如 ResNet、DenseNet）的常见做法。

5.5 对照实验设计

为系统评估不同因素对模型性能的影响，本文设计了三组对照实验：（1）BasicCNN + Adam：作为基准实验，检验基础 CNN 架构搭配 Adam 自适应优化器的性能表现；（2）BasicCNN + SGD：考察自适应优化器与带动量（Momentum=0.9）SGD 在该任务上的差异，SGD 学习率设为 0.01（初步测试认定为该任务下的合适值）；（3）Mini-ResNet + Adam：考察残差连接结构在

MNIST 任务上能否带来额外性能增益。三组实验除模型架构和优化器外，其余超参数均保持一致，确保对比的公平性。

六、实验结果与分析

6.1 定量评估

三组实验的验证准确率与最终测试准确率的对比结果如图 2 所示。从整体来看，三组实验均能有效收敛并达到 99.4% 以上的测试准确率：BasicCNN+Adam 收敛速度最快，在第 19 轮达到最佳验证准确率 99.33%；BasicCNN+SGD 初始收敛较慢但最终追平；Mini-ResNet+Adam 训练初期波动较大但最终稳定收敛，验证准确率达 99.27%。

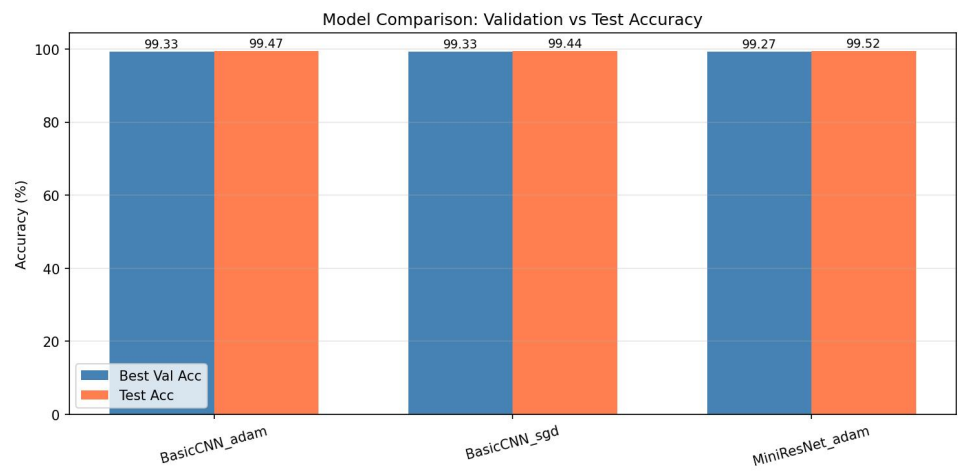


图 2 三组对照实验的验证与测试准确率对比（图片来源：根据实验数据由 ChatGPT 生成）

在测试集上的最终评估结果汇总如下。

表 3 三组对照实验性能对比

编号	模型	优化器	验证最优 Acc	测试 Acc	参数量
Exp-1	BasicCNN	Adam	99.33%	99.47%	390,858
Exp-2	BasicCNN	SGD	99.33%	99.44%	390,858
Exp-3	Mini-ResNet	Adam	99.27%	99.52%	178,634

由表 3 可知：（1）三组实验的测试准确率均超过 99.4%，表明 CNN 对 MNIST 任务具有极好的适应性，模型选择和优化器差异对最终性能的影响有限；（2）BasicCNN+Adam 的测试准确率（99.47%）高于验证最优值（99.33%），反映了良好的泛化能力，未见明显过拟合；（3）Mini-ResNet 以 99.52% 取得了最高的测试准确率，且参数量仅为 BasicCNN 的 45.7%，体现了残差连接和全局平均池化在提升参数效率方面的优势；（4）SGD 不仅收敛较慢，而且最终测试准确率与 Adam 仍差 0.03%，总的来说，对于该任务两种优化器均能取得优异结果，但 Adam 在训练效率上具有明显优势。

图 3 展示了 BasicCNN+Adam 模型在测试集上 10,000 个样本的混淆矩阵。可以直观看出，对角线元素占绝对主导，也就是说绝大多数样本被正确分类。错误分类主要集中于形态相似的数字对之间，如数字 4 与 9（部分手写体的顶部闭合程度相似）、数字 3、5 与 8（曲线笔画特征重叠）等，这些混淆模式与普通人的直觉判断高度一致。

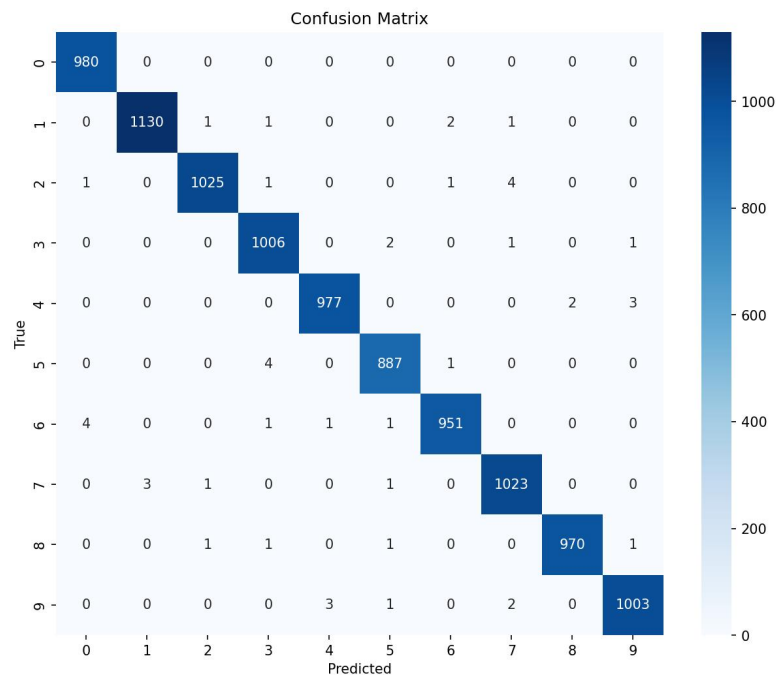


图 3 BasicCNN+Adam 在 MNIST 测试集上的混淆矩阵（图片来源：根据实验数据由 ChatGPT 生成）

表 4 列出了各类别的精确率（Precision）、召回率（Recall）和 F1 分数。所有类别的 F1 分数均超过 0.99，宏平均 F1 达到 0.9946，表明模型在各类别上的分类性能非常均衡，不存在明显的类别偏向。其中数字 2 的 F1 最高（0.9971），数字 9 的 F1 相对最低（0.9911），与混淆矩阵中数字 9 易与 4 混淆的观察一致。

表 4 BasicCNN+Adam 各类别分类性能详细统计

类别	Precision	Recall	F1-Score	Support
0	0.9939	0.9990	0.9964	980
1	0.9965	0.9974	0.9969	1,135
2	0.9961	0.9981	0.9971	1,032
3	0.9951	0.9960	0.9955	1,010
4	0.9919	0.9939	0.9929	982
5	0.9899	0.9933	0.9916	892
6	0.9989	0.9906	0.9948	958
7	0.9913	0.9961	0.9937	1,028
8	0.9990	0.9938	0.9964	974
9	0.9940	0.9881	0.9911	1,009
Macro Avg	0.9947	0.9946	0.9946	10,000

6.2 定性评估

为了更直观地理解模型的分类表现，图 4 展示了 BasicCNN 模型在测试集上的部分正确分类样本（绿色标注）。可以看出，模型对不同书写风格、笔画粗细和倾斜角度的数字均能准确识别，展现了良好的鲁棒性。即使是笔画断续、连笔或部分遮挡的数字，模型仍能正确分类。

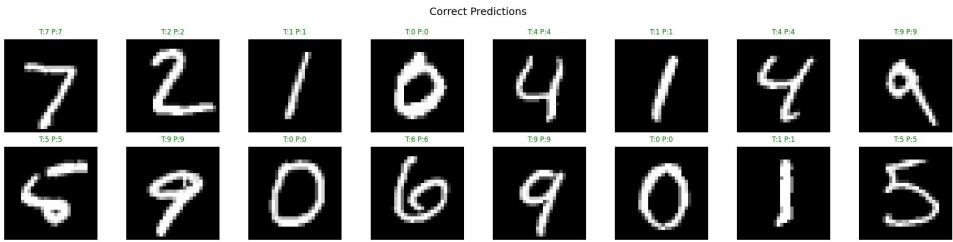


图 4 模型正确分类样本（绿色：真实标签 = 预测标签）（图片来源：使用 Matplotlib 从 MNIST 数据集中提取后显示）

图 5 展示了模型的部分错误分类案例（红色标注）。通过分析 16 个典型错误案例，可归纳出以下三类主要错误来源：（1）类间形态高度相似：如手写体“4”的顶部闭合使其与“9”难以区分，“3”的连笔写法与“8”混淆；（2）书写极端不规范：过度倾斜、笔画严重变形或含有异常断笔的数字超出了训练分布范围；（3）样本固有歧义：部分样本即使对于人类观察者也难以确定其真实类别。这些分析为后续模型改进提供了明确方向——可通过增强数据增强的多样性（如弹性变形、随机透视变换等）或引入注意力机制使模型聚焦于数字的判别性笔画区域，从而缓解特定类对之间的混淆。

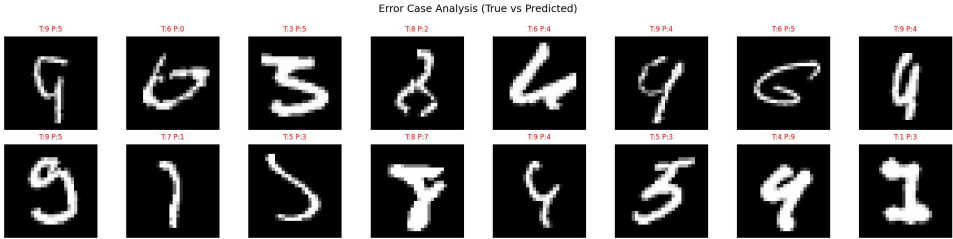


图 5 模型错误分类样本（红色：真实标签 → 预测标签）（图片来源：使用 Matplotlib 从 MNIST 数据集中提取后显示）

6.3 特征可视化

为直观理解 CNN 内部的特征提取机理，图 6 展示了 BasicCNN 第一层卷积核（32 个 3×3 核）的可视化结果。不同的卷积核呈现出各异的检测模式：有的倾向于检测水平边缘（亮-暗或暗-亮过渡），有的检测垂直边缘，还有的呈现对角方向的 Gabor 式纹理模式。这些多样化的卷积核各自负责检测输入图像中的特定局部特征，共同构成了网络最底层的特征检测基础。值得注意的是，这些卷积核是从数据中自动学习得到的，而非人工设计的滤波器。

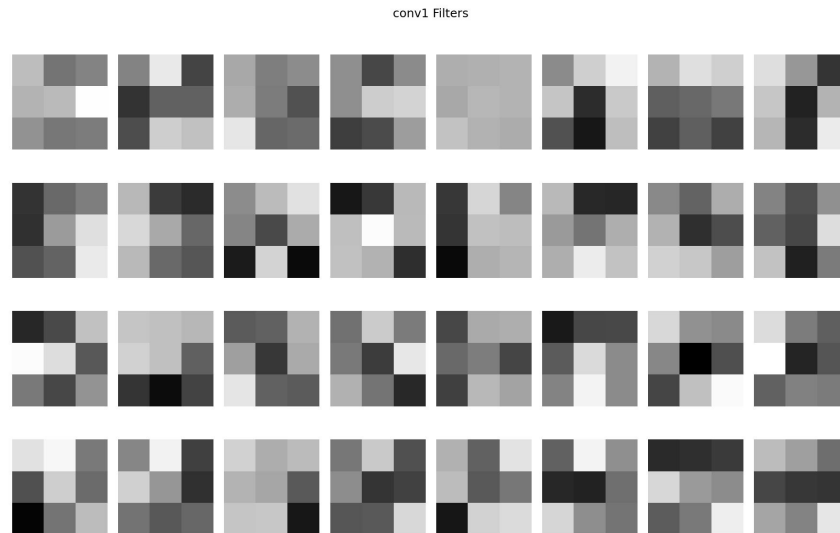


图 6 BasicCNN 第一层卷积核可视化（32 个 3×3 滤波器）（图片来源：实验训练结果提取）

图 7 进一步展示了输入数字"7"经过三个卷积层后的特征图变化。从 Conv1 到 Conv3，特征图的抽象程度逐步提升：Conv1 的特征图保留了较多的空间细节，能够清晰分辨笔画边缘和轮廓；Conv2 的特征图开始呈现更抽象、更全局的纹理模式，局部细节逐渐模糊；Conv3 的特征图则高度抽象化，不同的通道对应不同的高级语义特征响应，原始像素信息已完全转化为语义级别的特征表示。这一层次化的特征提取过程直观验证了 CNN"低级特征→中级特征→高级语义"的核心设计思想。

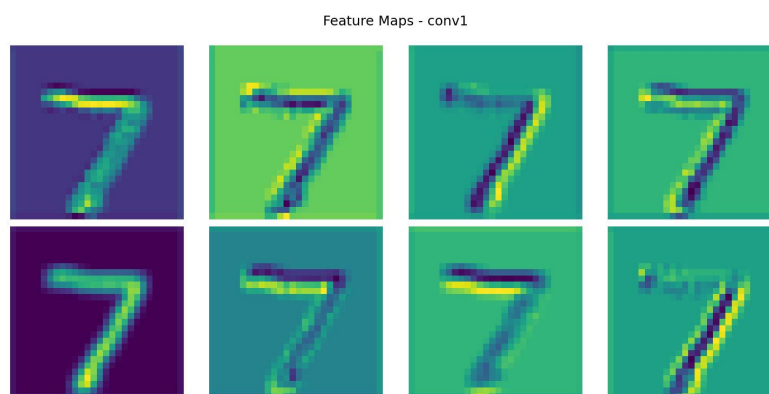


图 7(a) Conv1 层输出特征图（图片来源：实验测试结果生成）

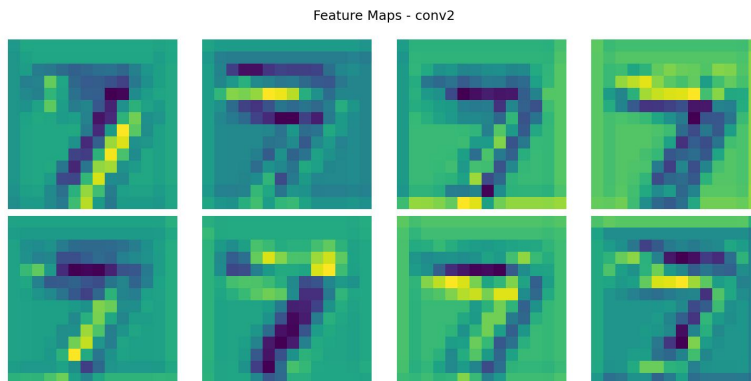


图 7(b) Conv2 层输出特征图（图片来源：实验测试结果生成）

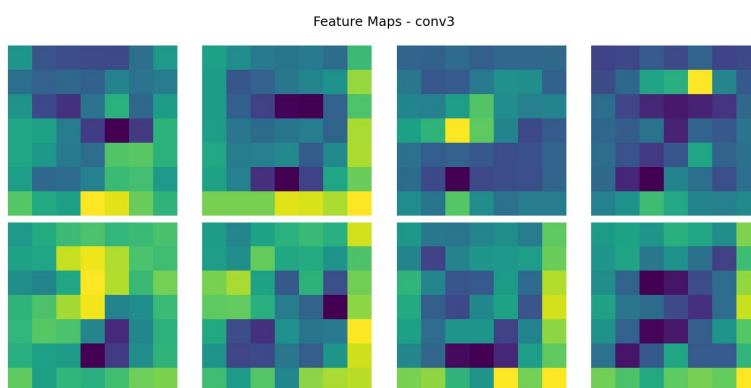


图 7(c) Conv3 层输出特征图（图片来源：实验测试结果生成）

6.4 讨论

综合三组对照实验的定量与定性分析结果，可得出以下结论：

（1）架构选择：BasicCNN 和 Mini-ResNet 在 MNIST 任务上的性能高度接近（测试准确率差距仅 0.05%）。这表明对于 28×28 灰度图像这类相对简单的视觉任务，适度的模型容量（约 20–40 万参数）已足够充分拟合数据分布，更深、更复杂的架构未必带来显著的性能增益。然而，Mini-ResNet 以不到一半的参数量达到同等甚至略优的性能水平，证实了残差连接和全局平均池化在参数效率方面的优势——用更少的参数达到了相同甚至更好的结果。

（2）优化器选择：Adam 在训练初期收敛速度明显优于 SGD，能够在更少轮数内达到较高准确率，体现了自适应学习率机制的优势。但 SGD 经过充分训练后

（约 15 轮）达到了几乎相同的泛化性能（99.44% vs 99.47%），这与近年来的研究结论一致：自适应优化器虽然收敛更快，但最终泛化性能不一定优于精心调参的 SGD。对于资源受限或需要快速迭代的场景，Adam 是更高效的选择；对于追求极致性能的场景，SGD 配合充分的训练和适当的学习率调度策略仍然值得考虑。

（3）局限与改进方向：尽管所有模型均取得超过 99.4% 的测试准确率，但错误案例分析揭示了模型在极端不规范书写和类间形态高度相似场景下的不足。未来的改进方向包括：引入更丰富的数据增强策略（弹性变形、随机擦除 Cutout、Mixup 等）；尝试集成学习（Ensemble）方法融合多个模型的预测；引入注意力机制（如 CBAM、SENet）增强模型对判别性特征区域的关注；以及探索更现代的架构设计（如 MobileNet 中的深度可分离卷积）在保持高性能的同时进一步减少参数量。

七、结论

本文基于 PyTorch 深度学习框架，完成了卷积神经网络在 MNIST 手写数字识别任务上的完整复现实验。论文系统梳理了 CNN 的核心原理（卷积层、池化层、激活函数、全连接层、Softmax 分类器及反向传播算法），设计并实现了 BasicCNN 和 Mini-ResNet 两种模型架构，通过三组对照实验（BasicCNN+Adam、BasicCNN+SGD、Mini-ResNet+Adam）全面评估了不同架构和优化策略对分类性能的影响。

实验结果表明，CNN 模型在 MNIST 任务上表现优异，三组实验的测试准确率均超过 99.4%，其中 Mini-ResNet+Adam 取得了最高的 99.52% 测试准确率。通过混淆矩阵分析和分类报告，验证了模型在各类别上的均衡表现（宏平均 $F1=0.9946$ ）；通过错误案例的定性分析，识别了模型的典型失效模式并提出了针对性改进方向；通过卷积核和特征图的可视化，直观展示了 CNN 层次化特征提取的内在机理。

通过本次课程论文的撰写，本人在理论与实践两个层面均取得了显著收获：在理论层面，系统巩固了从线性回归、对数几率模型、多层感知机到卷积神经网络

络的核心知识体系，加深了对深度学习基本原理和数学基础的理解；在实践层面，掌握了使用 PyTorch 框架进行模型搭建、训练、调优及评估的完整流程，积累了深度学习的实际工程经验。从学期初大一学生误闯天家，连 Python 语法都尚未熟悉，到逐渐理解多层感知机，最终大致清楚卷积神经网络的基本原理，本人认为习得的这些知识和技能应该会为今后从事更复杂的人工智能研究与应用工作奠定坚实基础。

参考文献

- [1] 杨梓铎. 人工智能课程讲义（尤其是 Lesson7、Lesson9、Lesson10）[Z]. 暨南大学电子工程系, 2026.
- [2] 不懂程序的开发猿. 基于卷积神经网络的手写数字识别 [EB/OL].
https://blog.csdn.net/qq_44807756/article/details/138193484, 2026-06
- [3] Weixin_55008315. PyTorch 实战：从零搭建手写数字识别系统（CNN 卷积神经网络）[EB/OL].https://blog.csdn.net/weixin_55008315/article/details/161363316?spm=1001.2014.3001.5502_2, 2026-06
- [4] 诺亚方包. PyTorch——手写数字识别 [EB/OL].
<https://wao666.blog.csdn.net/article/details/119980543?spm=1001.2014.3001.5502>, 2026-06
- [5] PyTorch. PyTorch Tutorial（尤其是 learn the basics 部分）[EB/OL].
<https://docs.pytorch.org/tutorials/index.html#>, 2026-06
- [6] 程勇 uestc. 手把手用 keras 分类 mnist 数据集[EB/OL].
<https://chengyong.blog.csdn.net/article/details/105846692>, 2026-06
- [7] 谦行 Aling. 识别手写数字——为什么你不该死磕 PyTorch 语法[EB/OL].
https://www.bilibili.com/video/BV1uPGj6fEQh/?spm_id_from=333.1387.favlist.content.click&vd_source=c698ea57bce3adf7e11681db2d48e70d, 2026-04
- [8] 天理难容 hhhhh. 手把手完成 mnist 手写数字识别[EB/OL].
https://www.bilibili.com/video/BV16g4y1z7Qu/?spm_id_from=333.1387.favlist.content.click&vd_source=c698ea57bce3adf7e11681db2d48e70d, 2026-04

[9] 孔工码子. 10 分钟入门神经网络 PyTorch 手写数字识别[EB/OL].

https://www.bilibili.com/video/BV16g4y1z7Qu/?spm_id_from=333.1387.favlist.content.click&vd_source=c698ea57bce3adf7e11681db2d48e70d , 2026-05

[10] Goodfellow I, Bengio Y, Courville A. Deep Learning[M]. Cambridge, MA: MIT Press, 2016.

[11] ChatGPT

[12] Codex